

Course Title:	Cloud Distributed Computing
Course Number:	COE892
Semester/Year:	W2026

Instructor:	Dr. Muhammad Jaseemuddin
-------------	--------------------------

Assignment/Lab Number:	Lab 4-5
Assignment/Lab Title:	FastAPI and Containers

Submission Date:	2026-03-31
Due Date:	2025-04-03

Student Last Name:	Student First Name:	Student Number:	Section:	Signature:
Tang	Derek	501177505	04	

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab reports your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at:

<http://www.ryerson.ca/senate/current/pol60.p>

Introduction

This lab report covers the design and implementation of a RESTful HTTP server with the use of FastAPI and a HTTP interface to simulate a rover and mine management system. Communication through the server and operators is done through JSON. The system was containerized through Docker and deployed through Microsoft Azure applications.

Part 1

The server was created through the FastAPI in Python and implements all required endpoints. A route called “/docs” can be accessed at <https://coe892lab42022g501177505-f8ftb9ccf5hpgdax.canadacentral-01.azurewebsites.net/docs> or locally for testing purposes.

- GET /map, PUT /map: retrieve and resize the mine field grid
- GET /mines, POST /mines, GET/PUT/DELETE /mines/{id}: full CRUD for mines
- GET /rovers, POST /rovers, GET/PUT/DELETE /rovers/{id}: full CRUD for rovers
- POST /rovers/{id}/dispatch: simulates rover movement, mine detection, PIN cracking, and returns the traversal path

The mines and map files are loaded upon startup. Dispatch logic allows for rovers to execute left, right, move, and dig commands, and performs SHA-256 brute force operations. Rovers can be in four different states: Not started, moving, finished, or eliminated.

Part 2

The operator was created as a single HTML file that connects to the server via Fetch API. The interface supports the following functions:

- Viewing the 10x10 map file
- Adding and deleting mines
- Creating rovers with imputed commands
- Viewing dispatch results
- Timestamped log of server responses
- Configurable server URL (supporting both local and azure deployments)

Ground Control

Server:

Sync

Map

*	*	*	*	*	*	*	*	*	*
*	•	*	*	*	•	*	*	*	*
*	*	*	*	*	*	*	*	*	*
•	*	*	*	*	*	•	*	*	*
*	*	*	*	*	*	*	*	*	*
*	•	*	*	*	*	*	*	*	*
*	*	*	*	*	*	*	*	*	*
*	*	*	*	*	*	*	*	*	*
*	*	*	*	*	*	*	*	*	*
*	*	*	*	*	*	*	*	*	*

Mines

ID	Row	Col	Serial	
1	1	1	M001	Delete
2	1	5	M003	Delete
3	3	0	M004	Delete
4	5	1	M006	Delete

Add mine: Row

Col

Serial

Add

Rovers

ID	Status	
1	Eliminated	Dispatch Delete

Create rover: Commands

Create

Dispatch

Rover ID

Dispatch

Log

3:19:53 PM Map loaded 3:19:53 PM 5 mines loaded 3:19:53 PM 1 rovers loaded 3:19:55 PM Selected cell (0,0) 3:20:00 PM Mine 5 deleted 3:20:00 PM Map loaded 3:20:00 PM 4 mines loaded 3:20:00 PM 1 rovers loaded	
---	-------------

Figure 1. Operator Interface

Part 3

Part 3 of the lab was skipped as it was optional.

Part 4

The server was containerized using Docker and a Dockerfile. The initial container was set up, deployed and pushed to the remote container registry.

```
PS C:\Users\darkt> docker tag lab4-5 COE892Lab42022G501177505.azurecr.io/lab4-5:latest
PS C:\Users\darkt> docker push COE892Lab42022G501177505.azurecr.io/lab4-5:latest
The push refers to repository [COE892Lab42022G501177505.azurecr.io/lab4-5]
e38be7ef3dd0: Pushed
e35b92e2619d: Layer already exists
6944007a5b45: Layer already exists
ad663a8a49bd: Pushed
a4e8b2ba5278: Pushed
7524aba74b8c: Layer already exists
ce370fc2a812: Layer already exists
ec781dee3f47: Layer already exists
91f4ffd50c9c: Pushed
latest: digest: sha256:ab61b501739de9b6b0e83b11ad88455c11874aad0ec89c93d4ffef566e6e845 size: 856
```

Figure 2. Containerization

Results

Below is a simulation of a rover with commands “MMMM” running and encountering a mine. Upon encountering the mine the rover is eliminated and results are printed in the server log.

Rovers

ID	Status	
1	Eliminated	<button>Dispatch</button> <button>Delete</button>

Create rover: Commands Create

Dispatch

Rover ID Dispatch

```
* 0 0 0 0 0 0 0 0 0
* 0 0 0 0 0 0 0 0 0
* 0 0 0 0 0 0 0 0 0
* 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
```

Log

```
6:39:34 PM Error: Rover 1 not found
6:39:42 PM Rover created ID=1
6:39:42 PM Map loaded
6:39:42 PM 0 mines loaded
6:39:42 PM 1 rovers loaded
6:39:48 PM Rover 1 -> Eliminated at (3,0)
6:39:48 PM Map loaded
6:39:48 PM 0 mines loaded
6:39:48 PM 1 rovers loaded
```

Figure 3. Simulation results

Conclusion

Overall the lab was a success as all three parts of the lab were fully functional. The FastAPI server handles all mapping, mine and rover operations via REST API. The operator was built through HTML and provides a simple user interface connecting to the server. The application was containerized through Docker and deployed to Microsoft Azure making it accessible publicly.